



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

计算机科学与工程系

工业实习报告

学生姓名：高进

学号：12212618

专业导师：张殷乾

实习单位指导老师：刘城城

实习单位名称：深圳迅策科技股份有限公司

2026 年 5 月 28 日

报告内容（主要包括：实习时间和地点。实习工作内容：1）从事的具体实际问题；2）怎样将理论与实践融合解决了问题；3）掌握的技能 and 熟悉的业务；4）锻炼了哪些能力。实习感悟和经验总结等）

一、实习时间与地点

实习时间：2026 年 3 月 24 日至 2026 年 5 月 23 日，共计两个月

实习地点：深圳迅策科技股份有限公司

深圳迅策科技股份有限公司是一家专注于智慧能源管理领域的高新技术企业，公司主要从事新能源电站智能化管理系统的研发与服务。公司产品涵盖光伏电站、储能电站、风力电站等多种新能源场景的监控、运维、数据分析等功能。我在实习期间主要参与了公司智慧能源运维管理平台的后端开发工作。

二、实习工作内容

（一）从事的具体实际问题

在为期两个月的实习期间，我作为后端开发实习生，主要负责智慧能源运维管理平台的后端开发工作。该平台是一个面向新能源电站的综合性管理系统，包含设备监控、告警管理、工单处理、数据分析、报表统计等多个功能模块。系统采用前后端分离架构，后端基于 Spring Boot 框架开发，提供 RESTful API 接口供前端调用。

在实习过程中，我主要负责并解决了以下具体问题：

1. 统一工单管理模块的字段命名规范问题

平台的工单管理模块需要处理三种类型的工单：故障工单、巡检工单和临时工单。在项目初期，由于缺乏统一的字段命名规范，这三种工单的数据表结构存在不一致的问题。具体表现为：临时工单使用 `task_content` 字段存储工单描述内容，巡检工单使用 `result` 字段存储巡检结果，而故障工单使用 `description` 字段存储故障描述。这种不一致导致了几个问题：首先，前端开发人员需要针对不同工单类型编写不同的字段映射逻辑，增加了前端代码的复杂度；其次，服务端在进行跨类型工单查询时，需要编写额外的转换逻辑；最后，后续维护人员理解代码时容易产生混淆。

为了解决这个问题，我与项目经理进行了沟通，确认了统一字段命名的方案。选择将所有工单类型的描述字段统一命名为 `description`。在具体实施过程中，我首先修改了三个实体类（`TemporaryOrder`、`InspectionOrder`、`FaultOrder`），将相应的字段名统一；其次修改了服务实现类中所有引用这些字段的地方，包括查询逻辑、插入逻辑、更新逻辑等；然后在 `smart_energy.sql` 数据库脚本末尾添加了 `ALTER TABLE` 语句，用于修改临时工单表和巡检工单表的字段名称；最后修改了 `sample_data.sql` 样例数据脚本，确保插入的数据使用正确的字段名。

这个问题的解决让我深刻认识到，良好的数据库设计规范和一致的字段命名对于项目的长期维护至关重要。在实际项目中，往往由于进度压力或历史原因，会产生各种不一致的设计，作为开发人员，需要在合适的时候提出问题并推动改进。

2. 工单列表接口的派单人信息显示问题

在工单管理功能中，有一个重要的使用场景：当用户创建一个工单并指定处理人后，需要在工单列表中显示该处理人的基本信息，包括处理人姓名和所属组别。用户反馈在创建工单并选择处理人后，调用工单列表接口，却发现返回的数据中缺少处理人及其所属组别信息，导致前端页面上无法正常展示。

我首先查阅了相关代码，发现创建工单的 `create` 方法中，虽然设置了处理人的 `userId` 和 `userName` 字段，但没有设置 `groupId` 和 `groupName` 字段。经过进一步分析代码逻辑，我发现当工单被派发给某个处理人时，应该同时记录该处理人所属的用户组信息，这个信息对于前端展示和后续的派单决策都很重要。

针对这个问题，我在工单创建的逻辑中增加了处理人组别信息的设置。具体来说，当设置 `userId` 和 `userName` 时，额外查询该用户所属的组别信息，并将 `groupId` 和 `groupName` 一并设置到工单实体中。这样后续无论是列表查询还是详情查询，都能正确返回处理人的组别信息。

这个问题的解决过程让我认识到，在设计接口时需要充分考虑前端展示的需求。有时候后端开发人员认为不重要的字段，对于前端来说可能是必需的信息。良好的沟通和站在用户角度的思考非常重要。

3. 告警通知方式名称的本地化问题

平台的告警管理功能支持多种通知方式，包括短信（`sms`）、邮件（`email`）、微信（`wechat`）、电话（`phone`）以及站内消息（`message`）等。在实际使用中，用户反馈接口返回的通知方式显示为英文标识，例如页面上显示的是“`message`”而不是中文的“消息”，这严重影响了用户体验。

经过代码分析，我发现问题出在 `SystemConstant` 类中。该类中定义了大量的系统常量和方法，用于在不同地方复用，其中 `getAlarmTypeName` 方法用于获取告警类型的中文名称，`getDeviceTypeName` 方法用于获取设备类型的中文名称，但缺少获取通知方式中文名称的方法。

我在 `SystemConstant` 类中添加了 `getNotifyTypeName` 方法，该方法接收通知方式的英文标识作为参数，根据不同的标识返回对应的中文名称。例如，接收到“`sms`”时返回“短信”，接收到“`email`”时返回“邮件”，接收到“`wechat`”时返回“微信”，接收到“`phone`”时返回“电话”，接收到“`message`”时返回“消息”。然后修改了所有返回通知方式的地方，统一调用这个新方法进行转换。

这个问题的解决相对简单，但让我认识到用户体验往往体现在这些细节之处。一个系统的功能再强大，如果用户界面显示不友好，就会严重影响用户的使用意愿。

4. 工单类型筛选功能失效问题

平台的工单管理模块提供了强大的筛选功能，用户可以根据工单类型（`orderType`）、工单状态（`status`）、时间范围等多个维度对工单进行筛选。其中工单类型是一个重要的筛选维度，故障工单的 `orderType` 为 1，巡检工单的 `orderType` 为 2，临时工单的 `orderType` 为 4。

用户反馈在测试时发现，当使用 `orderType=2` 进行筛选时，接口返回的结果中包含了所有类型的工单，而不仅仅是巡检工单。这个问题严重影响了对特定类型工单的查询效率。

我通过添加日志和断点调试，发现问题出在工单类型的比较逻辑上。代码中使用了 `==` 运算符进行 `orderType` 的比较，而不是使用 `equals` 方法。由于 `orderType` 参数在传入时可能已经是包装类型 `Integer`，而比较时可能存在类型转换的问题，导致比较结果不正确。我将所有 `orderType` 的比较从 `==` 改为了 `equals` 方法，确保比较的准确性。

同时，在检查代码过程中，我还发现 `queryInspectionOrders` 方法和 `buildInspectionOrderSpec` 方法中引用了错误的字段名。原来 `inspection_order` 表中已经将 `result` 字段改名为 `description`，但某些查询逻辑中仍然使用 `result` 作为字段名进行查询，导致查询结果不准确。我将这些地方的字段名都修正为 `description`。

这个问题让我认识到，在进行数据查询时，一定要确保实体类字段名与数据库表字段名的一致性，以及查询条件中引用的字段名与实际字段名的一致性。

5. 仪表盘发电量数据的真实性问题

领导在查看平台仪表盘时，发现 `/dashboard/hourly-power-load` 接口返回的发电量数据不够真实。具体表现为：夜间（19:00-05:30）时段，光伏设备的发电功率仍然显示有值，这明显不符合光伏发电的物理特性，因为夜间没有阳光，光伏组件无法发电。

我首先检查了 `sample_data.sql` 中的样例数据，确认已经将光伏设备在夜间的 `power` 值设置为 0。但问题仍然存在，于是我进一步分析了接口的实现逻辑。

经过分析，我发现出问题出在接口实现上。该接口在计算发电量时，会对 `device_real_data` 表中所有设备的 `power` 字段进行求和。而 `device_real_data` 表中不仅包含光伏发电设备（`device_type=3`），还包含其他类型的设备，如电池簇（`device_type=5`）、配电柜（`device_type=2`）、变压器（`device_type=6`）等。这

些设备虽然不是光伏发电设备，但它们的 `power` 字段可能有值（例如电池的充放电功率、配电柜的负载功率等），这些值被错误地计入了发电量中。

为了解决这个问题，我在 `getHourlyPowerAndLoadData` 方法中增加了设备类型过滤逻辑。首先获取所有光伏发电主设备（`device_type=3`）的 ID 列表，然后在统计 `power` 时，只统计这些光伏设备的功率数据。这样就确保了 `hourlyPower` 字段只反映光伏发电的真实发电量，而不会混入其他设备的功率数据。

同时，我还修改了 `sample_data.sql` 样例数据，确保光伏设备在夜间（19:00-05:30）的 `power` 值为 0，中午时段（09:00-15:00）达到峰值，形成符合物理规律的发电曲线。

这个问题让我认识到，在进行数据统计时，需要充分考虑业务场景和数据来源的准确性。不同类型的设备可能使用相同的字段存储不同的物理量，统计时需要根据设备类型进行区分。

6. 数据库字段迁移问题

在项目开发过程中，数据库表结构可能需要根据业务变化进行调整。当我在修改实体类定义后（如将 `task_content` 改为 `description`），直接运行程序时出现了 `"Unknown column 'result' in 'field list'"` 的错误。这是因为数据库表中的字段名仍然是旧的，没有同步更新。

为了解决这个问题，我学习了如何在不丢失数据的情况下修改数据库表结构。在 `smart_energy.sql` 数据库初始化脚本的末尾，我添加了 `ALTER TABLE` 语句，用于在表创建后修改字段名称。例如：`ALTER TABLE inspection_order CHANGE COLUMN result description TEXT COMMENT '巡检结果/描述'`；`ALTER TABLE temporary_order CHANGE COLUMN task_content description TEXT NOT NULL COMMENT '任务内容/描述'`。

这样，当数据库初始化脚本执行时，会先创建表结构，然后执行 `ALTER TABLE` 语句进行字段重命名，既保留了原有的数据，又使表结构与实体类定义保持一致。

这个经历让我认识到，数据库表结构变更需要谨慎处理，特别是在有生产数据的情况下。在实际项目中，通常需要编写数据迁移脚本，并进行充分的测试。

（二）怎样将理论与实践融合解决了问题

在解决上述问题的过程中，我深刻体会到了理论知识与实践经验相结合的重要性。

1. 数据库设计的规范性理论得到了实践验证

在学校的数据库课程中，我们学习了数据库设计的基本原则，包括命名规范、

数据一致性、规范化设计等。巡检工单、临时工单、故障工单表结构不一致的问题，实际上就是数据库设计规范性问题。通过解决这个问题，我深刻认识到良好的数据库设计对于系统的可维护性和可扩展性至关重要。只有在设计阶段充分考虑各种情况，才能避免后期的频繁修改。

2. RESTful API 设计原则在实际项目中的应用

课堂上学到的 RESTful API 设计规范要求接口命名清晰、参数明确、返回结构一致。在统一工单模块的过程中，我体会到前后端分离开发中 API 一致性的重要性。当三个工单类型使用不同的字段名时，前端需要为每种类型编写不同的处理逻辑，增加了开发成本和维护难度。通过统一字段名和返回结构，大大简化了前端的工作。这让我认识到，良好的 API 设计不仅仅是后端的事情，更需要站在前端和用户的角度思考问题。

3. 软件工程的质量意识在实践中的体现

学校的软件工程课程教会了我软件生命周期、测试驱动开发、持续集成等概念。虽然在实际项目中由于进度压力，不可能做到完美的测试驱动开发，但每次修改代码后，我都会进行自测，确保修改没有引入新的问题。同时，使用 Maven 进行项目构建、使用 Git 进行版本控制，这些在学校学到的工具在实际工作中发挥了重要作用。

4. 问题诊断与调试能力的提升

在学校做实验时，遇到问题往往有标准答案或老师指导。而在实际项目中，很多问题没有现成的答案，需要自己探索。通过查看日志文件、使用 Postman 测试接口、在 IDE 中进行断点调试等手段，我的问题诊断和调试能力得到了很大提升。我养成了先分析问题、再定位原因、最后动手解决的习惯，而不是盲目尝试。

三、AI 辅助编程的经验与应用

（一）AI 辅助编程工具的使用经历

在实习期间，我有机会接触和使用 AI 辅助编程工具，这是我在学校学习期间没有过的全新体验。AI 辅助编程工具能够通过理解开发者的意图，自动生成代码片段、提供代码建议、甚至帮助分析和修复 bug。

在项目开发过程中，当遇到不熟悉的技术点或需要编写重复性较高的代码时，我会使用 AI 辅助编程工具来提高效率。例如，在编写数据库查询的 SQL 语句时，AI 工具能够根据我描述的需求自动生成相应的 SQL 代码，我只需要进行适当的调整和优化。

AI 辅助编程工具特别擅长处理那些模式化、模板化的代码任务。对于这类任务，

AI 能够快速生成高质量的代码，让我有更多时间专注于业务逻辑的思考和优化。

（二）AI 辅助编程在代码优化中的应用

在代码优化方面，AI 辅助编程工具也发挥了重要作用。有一次，我需要重构一段复杂的查询逻辑，AI 工具帮助我分析了现有代码的结构和潜在问题，并提出了优化建议。虽然最终的优化决策是由我做出的，但 AI 工具提供的分析视角让我对代码有了更深入的理解。

另一个例子是代码审查。当我完成一个功能的代码后，会使用 AI 工具进行静态代码分析，检查潜在的 bug、代码规范问题和安全漏洞。AI 工具能够发现一些我可能遗漏的问题，提高了代码质量。

（三）AI 辅助编程的局限性与注意事项

在使用 AI 辅助编程工具的过程中，我也认识到了它的局限性。首先，AI 生成的代码有时候并不完全符合实际需求，需要开发者进行仔细的审查和调整。其次，对于复杂的业务逻辑，AI 工具往往难以准确理解开发者的意图，生成的代码可能需要大量修改。

更重要的是，AI 辅助编程工具不能替代核心的编程能力和问题解决能力。它更像是一个高效的助手，能够帮助处理大量的重复性工作，但面对真正复杂的问题时，仍然需要开发者自身的技术能力和经验来判断和解决。

（四）对 AI 辅助编程的认识和思考

通过实习期间的使用，我认为 AI 辅助编程工具将会成为未来软件开发的重要辅助手段。它能够帮助开发者提高效率、减少低级错误、让开发者有更多时间专注于创造性的工作。但同时，我们也需要正确看待它的作用，不能过度依赖。

作为计算机专业的学生，我们应该积极拥抱新技术，学会使用 AI 辅助编程工具来提高自己的工作效率。但更重要的是打好基础，深入理解计算机科学的基本原理，因为这些基础知识是任何工具都无法替代的。

四、掌握的技能 and 熟悉的业务

（一）技术技能

1. Java 后端开发技术

通过实习，我熟练掌握了 Spring Boot 框架进行 Web 应用开发的能力。能够使用 Spring Boot 快速搭建 RESTful API 服务，理解了控制反转 (IoC) 和依赖注入 (DI) 的原理和实际应用。能够使用 @Autowired 注解或构造函数注入管理 Bean 依赖，使用 @RestController 和 @RequestMapping 等注解定义 API 接口。

2. 数据库技术

熟练掌握了 MySQL 数据库的使用，包括数据库创建、数据表设计、SQL 查询编写等。能够编写复杂的 SELECT 查询、INSERT 插入、UPDATE 更新和 DELETE 删除语句。理解了事务的概念和 ACID 特性，能够处理并发访问时的事务问题。掌握了 ALTER TABLE 语句进行表结构修改，理解了字段重命名和数据迁移的方法。

3. JPA/Hibernate ORM 框架

掌握了使用 JPA/Hibernate 进行对象关系映射开发。能够编写 Entity 实体类，通过 @Entity 和 @Table 注解映射数据库表，通过 @Id 和 @GeneratedValue 注解定义主键，通过 @Column 注解映射字段。理解了 JPA 的 Repository 接口和自定义查询方法的使用。能够编写 Specification 进行复杂条件查询。

4. API 开发与调试

熟练掌握了 RESTful API 的设计规范和开发流程。能够设计清晰的 API 接口路径、选择合适的 HTTP 方法（GET、POST、PUT、DELETE 等）、定义合理的请求参数和返回结构。熟练使用 Postman 等 API 测试工具进行接口测试，能够构造各种请求场景验证接口的正确性。

5. 开发工具使用

熟练使用 IntelliJ IDEA 作为 Java 开发 IDE，掌握了其代码补全、调试、版本控制集成等功能。掌握了 Maven 项目构建工具，能够创建项目、添加依赖、打包部署等。初步了解了 Git 版本控制系统，能够进行代码提交、拉取、推送等基本操作。

6. AI 辅助编程

初步掌握了使用 AI 辅助编程工具进行代码开发、代码审查和问题诊断。能够利用 AI 工具提高编码效率，但对 AI 生成的内容保持批判性思维，进行必要的审核和调整。

（二）业务知识

1. 智慧能源管理系统业务

通过实际参与项目，我对智慧能源管理系统有了深入的了解。了解了光伏电站、储能电站等新能源电站的基本构成和工作原理。理解了电站在实际运行中需要监控的各类参数，如发电功率、发电量、SOC（储能电池荷电状态）、温度等。认识到新能源电站运维管理的复杂性和重要性。

2. 工单管理系统业务

掌握了工单从创建、派单、处理、归档到作废的完整生命周期管理。理解了故障工单、巡检工单、临时工单三种工单类型的区别和适用场景：故障工单用于记录设备故障的发现、处理和解决过程；巡检工单用于安排和记录定期设备巡检任务；临时工单用于处理临时性的工作任务。理解了工单与告警、设备之间的关联关系。

3. 数据可视化与分析

了解了仪表盘数据展示的设计原则和实现方法。掌握了 24 小时发电量/负荷曲线的业务含义和展示要求。理解了不同设备类型（发电设备、储能设备、配电设备）数据的差异，以及如何根据业务需求进行数据筛选和统计。

4. 用户权限与组织架构

了解了系统中的用户管理、角色权限设计以及用户组概念。用户属于特定的用户组，用户组有组长和成员。工单可以派发给个人用户或用户组，理解了个人处理与团队协作在工单分配中的不同应用场景。

五、锻炼了哪些能力

1. 问题分析能力

面对复杂的系统问题，不再急于动手，而是先分解问题、逐步排查。通过分析错误日志、追踪代码逻辑、查阅文档等手段，能够较为准确地定位问题根源。这种系统化的问题分析方法大大提高了解决问题的效率。

2. 代码编写能力

通过大量的编码实践，代码编写能力得到了显著提升。更加注重代码规范，包括命名规范、缩进格式、注释编写等。理解了 DRY（Don't Repeat Yourself）原则，尽量抽取公共逻辑进行复用。代码质量意识显著增强。

3. 沟通协作能力

在实习期间，需要与项目经理、前端开发人员、测试人员等多个角色进行沟通。学会了准确理解需求方的意图，能够用简洁清晰的语言解释技术方案。在遇到不明确的需求时，主动向相关人员确认，避免做无用功。

4. 学习总结能力

每次解决一个问题后，都会进行总结记录。这种习惯不仅加深了对问题的理解，也方便以后遇到类似问题时快速回顾。同时，养成了查阅官方文档和搜索技

术博客的习惯，能够较快地学习新技术和新方法。

5. 时间管理能力

在实际工作中，往往需要同时处理多个任务。学会了根据任务的重要性和紧急程度进行优先级排序，合理安排工作时间。能够根据项目进度要求调整工作计划，确保按时交付。

六、实习感悟和经验总结

（一）实习感悟

两个月的实习虽然时间不长，但让我收获颇丰。这段时间不仅让我学到了很多实用的技术技能，更重要的是让我完成了从学生到职场人的心态转变。

首先，我深刻认识到了沟通的重要性。在学校里，很多课程作业都有明确的要求和标准答案，而工作中很多需求是不够明确的，需要与产品经理或需求方反复沟通确认。我一开始总是拿到需求就开始做，结果做出来的东西与预期不符，浪费了不少时间。后来我学会了先理解需求，用自己的话复述给需求方确认，遇到不确定的地方及时沟通，确保理解正确后再动手。

其次，我体会到了代码质量的重要性。学校的作业只需要跑通就行，代码规范和注释都不是考察重点。但实际工作中的代码需要考虑可维护性、可扩展性。一个好的命名、清晰的注释、合理的代码结构，能够让后来接手的人快速理解，也能让自己在一段时间后快速回忆起来。良好的代码规范是对团队负责，也是对自己负责。

第三，我认识到了持续学习的必要性。技术在不断发展，新的框架、新的工具层出不穷。在学校学到的知识可能在工作中只用到一小部分，但学校培养的学习能力和思维方式是最重要的。面对没有接触过的技术，我不再畏惧，而是敢于尝试，善于查阅文档和搜索资料，快速上手新技能。特别是 AI 辅助编程等新技术的出现，让我意识到保持学习热情和开放心态的重要性。

第四，我理解了团队协作的意义。一个完整的系统需要产品、前端、后端、测试等多个角色协作完成。作为后端开发，我不能只关注自己的代码实现，还要了解前端的实现方式、数据库的设计结构、测试的关注点。只有站在全局的角度思考，才能写出真正满足需求的接口，才能与团队成员高效协作。

第五，我感受到了职业责任的重要性。学校的作业做不好，最多影响自己的成绩；但工作中的代码质量不好，可能会影响整个团队的工作进度，甚至影响产品的质量。这种责任感让我在工作中更加认真仔细，尽量避免低级错误。

第六，我体验到了 AI 辅助编程为开发工作带来的变革。AI 工具能够大大提高编码效率，特别是在处理重复性高的任务时。但我也认识到，AI 只是辅助工具，不

能替代核心的编程能力和问题解决能力。扎实的计算机基础知识、良好的编程习惯、系统化的思维能力，这些才是程序员的核心竞争力。

（二）经验总结

通过这次实习，我总结了以下几点经验教训：

1. 动手之前先理解需求

不要急于写代码，首先要确保理解清楚需求。如果需求有歧义，一定要与产品或需求方确认。有时候一个小小的歧义可能导致整个方向错误，返工的成本远大于前期沟通的成本。建议在开始编码前，用文档或口头的形式与需求方确认对需求的理解。

2. 做好版本控制

每次修改前先提交当前版本，修改完成后再次提交并写清楚 **commit message**。这样如果修改出现问题，可以快速回退到之前的版本。特别是在修改较大功能时，版本控制是必不可少的保险措施。

3. 重视测试

不要以为代码写完就完了。主动进行自测，确保基本功能正常。主动与测试人员沟通测试计划，了解哪些场景需要重点测试。对于边界条件和异常情况的处理要特别关注。

4. 记录问题和解决方案

遇到的问题和解决方法要及时记录下来。这样做有几点好处：一来可以加深对问题的理解，二来以后遇到类似问题可以快速回顾，三来可以为团队其他成员提供参考。建议使用有道云笔记、**OneNote** 等工具进行记录。

5. 保持代码规范

遵循团队的代码规范，保持命名一致性，编写必要的注释。这不仅方便自己，也方便团队其他成员。如果团队没有明确的规范，建议查阅 **Google Java Style Guide** 等业界标准。

6. 善于使用工具

熟练使用各种开发工具可以大大提高效率。**IDE** 的快捷键、版本控制工具的命令行操作、数据库管理工具等，都值得花时间学习。同时，也要学会使用 **AI** 辅助编程工具来提高效率，但要保持批判性思维，对 **AI** 生成的内容进行审核。磨刀不误砍柴工，花时间学习工具的使用，长期来看会节省大量时间。

7. 理解业务比技术更重要

技术是为业务服务的。作为后端开发，不仅要关注代码实现，还要深入理解业务逻辑。只有深入理解业务，才能设计出真正满足需求的系统，才能在需求变更时快速做出调整，才能与产品人员有效沟通。

8. 正确看待 AI 辅助编程

AI 辅助编程是趋势，但要正确看待它的作用。它能够帮助处理重复性工作、提供代码建议、进行代码审查，但不能替代核心的编程能力。扎实的计算机基础知识、良好的编程习惯、系统化的思维能力，这些才是程序员的核心竞争力，无法被任何工具替代。

总之，这次实习是我职业生涯的重要起点。我会将这次实习中学到的知识和技能继续运用到今后的学习和工作中，不断提升自己的专业能力，为成为一名称职的软件工程师打下坚实基础。同时，我也会持续关注 AI 等新技术的发展，学会利用这些工具提高自己的工作效率，但始终保持对基础知识的重视和对技术的敬畏之心。

评分： 95

实习单位指导老师签字：刘城城

评分：

专业导师签字：

备注：

1. 语言：根据学校专业导师要求使用中文或英文撰写；
2. 字数：不少于 3000 字。